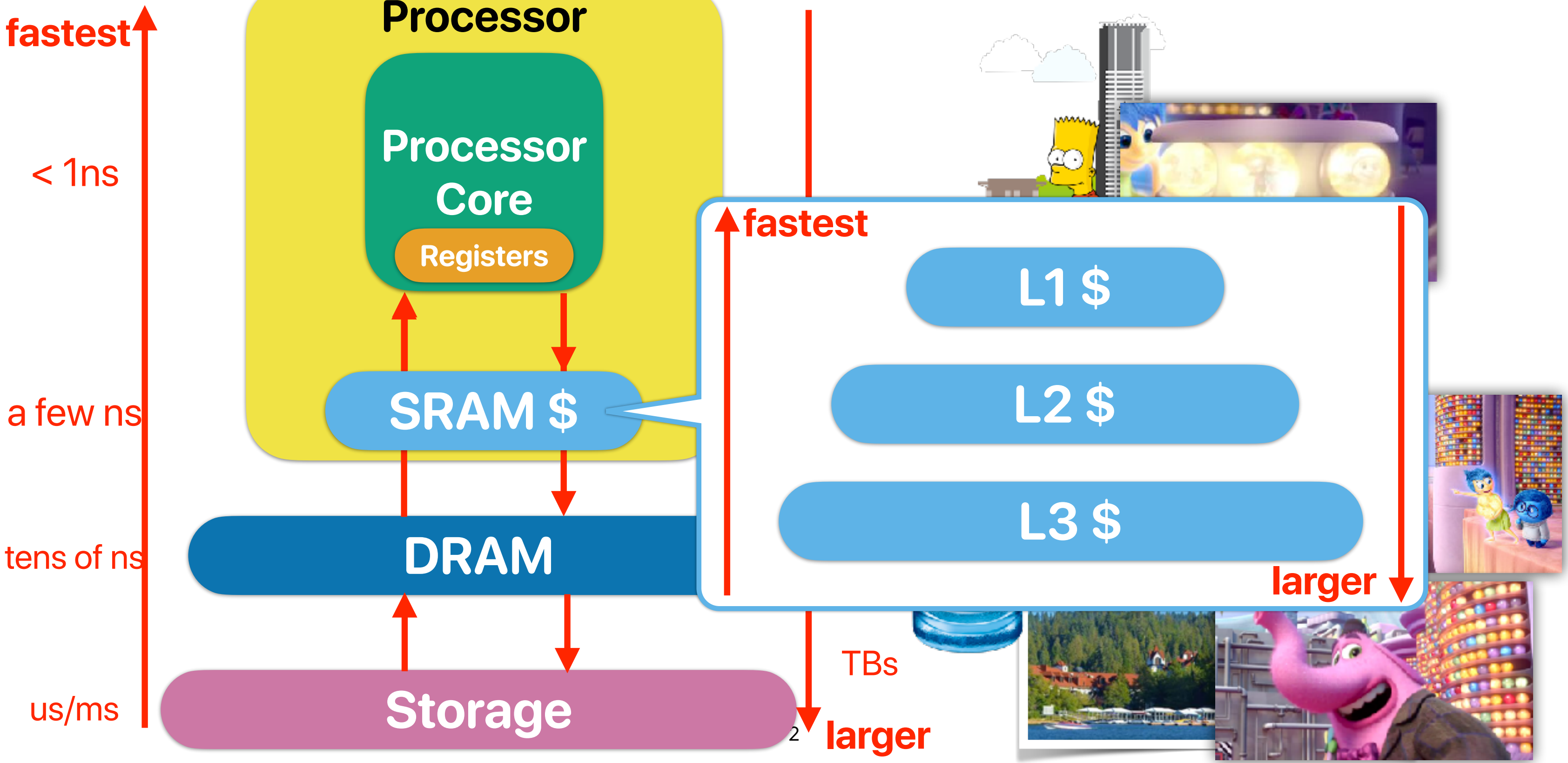


Lab 3: More aspects on memory

Hung-Wei Tseng

Recap: Memory Hierarchy



Recap: Software Optimizations

- Data layout — capacity miss, conflict miss, compulsory miss
- Loop interchange — conflict/capacity miss
- Loop fission — conflict miss — when \$ has limited way associativity
- Loop fusion — capacity miss — when \$ has enough way associativity
- Blocking/tiling — capacity miss, conflict miss
- Matrix transpose (a technique changes layout) — conflict misses
- Prefetching — compulsory misses

The concept of working set

Working set size

- The portion of memory that the program is currently using
- The total size of data blocks with “temporal locality”
- The total size of data blocks we have to visit before the next reuse of the current block

What's the working set size of the code?

- D-L1 Cache configuration of intel Core i7 processor
 - Size 48KB, 12-way set associativity, 64B block, LRU policy, write-allocate, write-back, and assuming 64-bit address.

```
double a[16384];
/* a = 0x20000 */
for(j = 0; j < runs; j++)
    for(i = 0; i < size; i+=stride) {
        sum += a[i];
        //load a, b, and then store to c
    }
}
```

What's the data cache miss rate for this code when **stride = 8**, **runs = 4** and **size = 1024**?

Let's pick the block with a[0], how many blocks do we have to visit before coming back?

We have to visit a total of $\frac{1024}{8} = 128$ *blocks*

We need $128 \times 64 = 8$ *KB* **capacity to keep the "working set"**

What's the working set size of the code?

- D-L1 Cache configuration of intel Core i7 processor
 - Size 48KB, 12-way set associativity, 64B block, LRU policy, write-allocate, write-back, and assuming 64-bit address.

```
double a[16384];
/* a = 0x20000 */
for(j = 0; j < runs; j++)
    for(i = 0; i < size; i+=stride) {
        sum += a[i];
        //load a, b, and then store to c
    }
}
```

What's the data cache miss rate for this code when **stride = 8**, **runs = 4** and **size = 8192**?

Let's pick the block with a[0], how many blocks do we have to visit before coming back?

We have to visit a total of $\frac{8192}{8} = 1024$ blocks

We need $1024 \times 64 = 64$ KB capacity to keep the "working set"

How to reduce the working set?

How a “struct” is layout?

Thinking in cache lines

- How big is my_struct?
 - 16 bytes
 - 4 uint32_t's
 - 1 cache line?
 - 2 cache lines?

```
struct my_struct {  
    uint32_t foo[4]  
}
```

Q4: The result of `sizeof(struct student)`

- Consider the following data structure:

```
struct student {  
    int id;  
    double *homework;  
    int participation;  
    double midterm;  
    double average;  
};
```

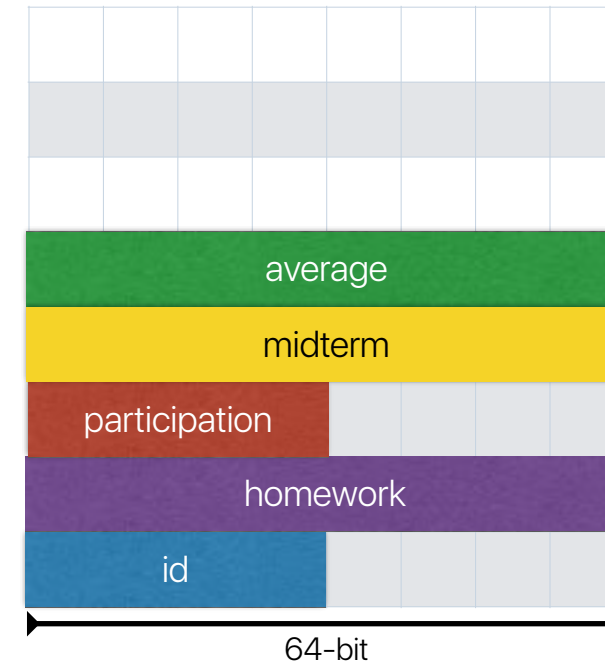
What's the output of
`printf( "%lu\n", sizeof(struct student) )`?

- A. 20
- B. 28
- C. 32
- D. 36
- E. 40

The result of `sizeof(struct student)`

- Consider the following data structure:

```
struct student {  
    int id;  
    double *homework;  
    int participation;  
    double midterm;  
    double average;  
};
```



What's the output of `printf( "%lu\n", sizeof(struct student) )`?

- A. 20
- B. 28
- C. 32
- D. 36
- E. 40**

Memory addressing/alignment

- Almost every popular ISA architecture uses “byte-addressing” to access memory locations
- Instructions generally work faster when the given memory address is aligned
 - Aligned — if an instruction accesses an object of size n at address X , the access is **aligned** if **$X \bmod n = 0$** .
 - Potentially incurs two cache misses for an access
 - Some architecture/processor does not support aligned access at all
 - Therefore, compilers only allocate objects on “aligned” address

Memory addressing/alignment

- Unaligned accesses are sometimes inefficient: one load can cause 2 cache misses
- Some versions of ARM's ISA have very complicated semantics for unaligned accesses
- Other ISAs
 - Unaligned access can cause an interrupt.

Compiler cannot optimize the layout

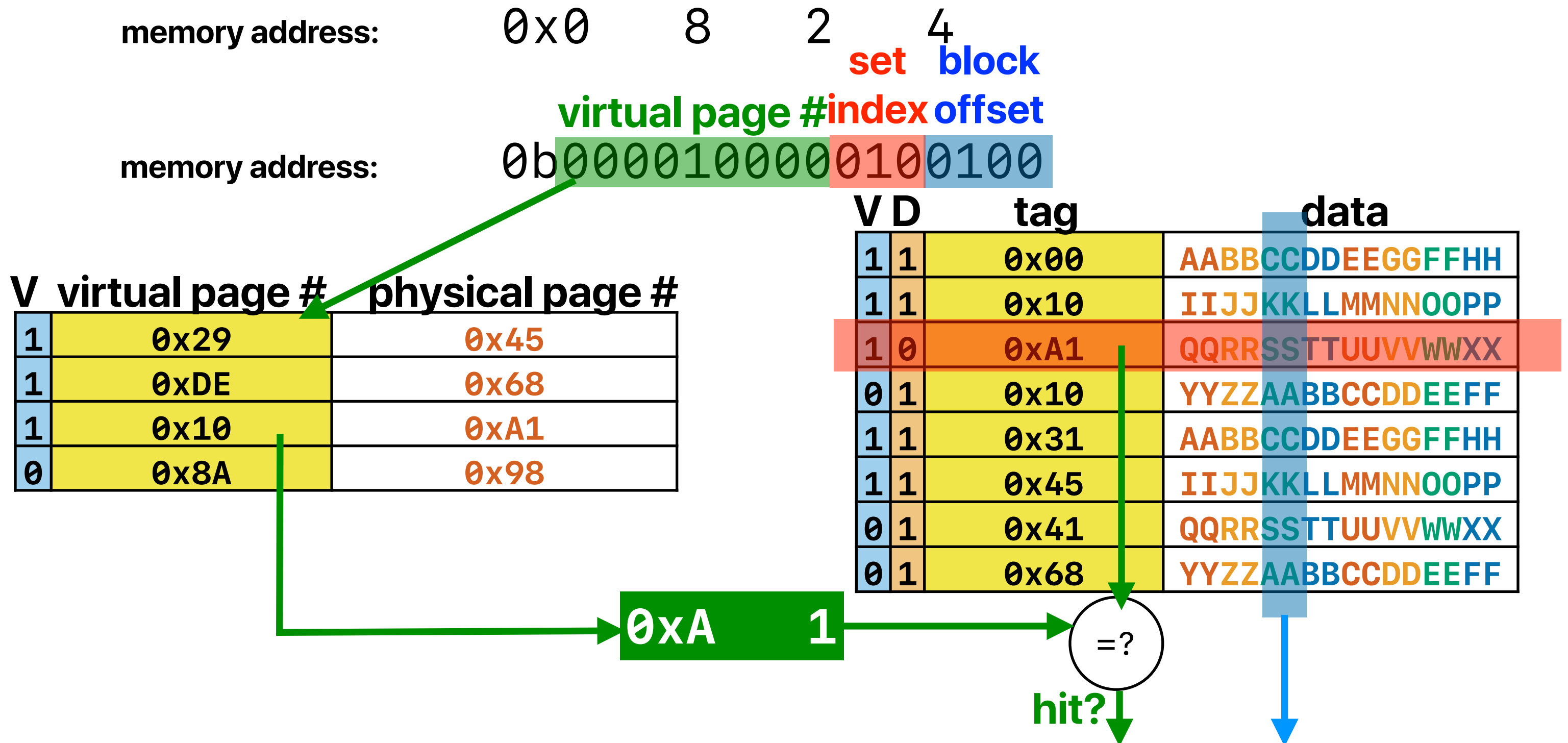
- The struct declaration must be shared among binaries use that struct — not everyone has the same optimization flags
- Accessing the struct element is base + offset in the struct
 - Rearranging without everyone's agreement would cause errors

Takeaways: Software Optimizations

- Data layout — capacity miss, conflict miss, compulsory miss
- Loop interchange — conflict/capacity miss
- Loop fission — conflict miss — when \$ has limited way associativity
- Loop fusion — capacity miss — when \$ has enough way associativity
- Blocking/tiling — capacity miss, conflict miss
- Matrix transpose (a technique changes layout) — conflict misses
- Prefetching — compulsory misses
- Data alignments — capacity misses

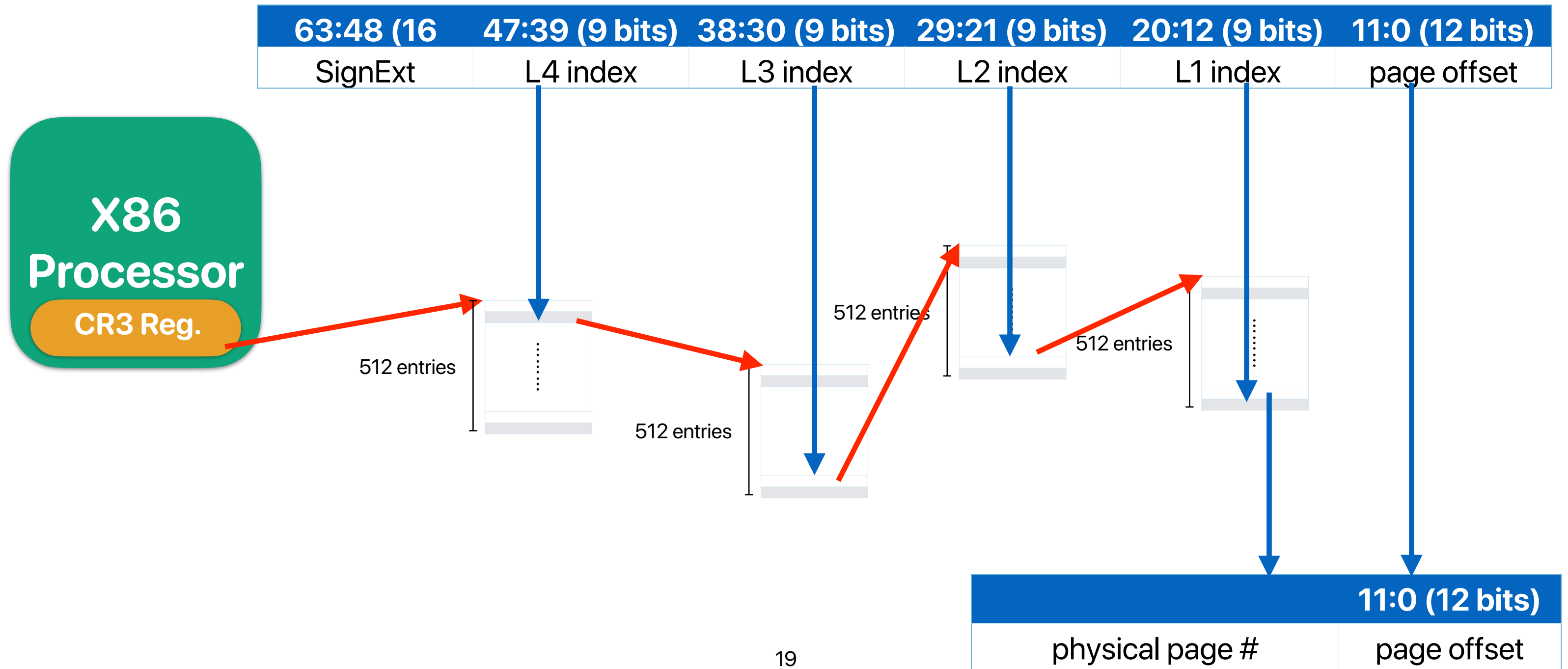
**Remember, we also have virtual
memory**

Memory hierarchy in practice: TLB + Cache



What happen on a TLB miss?

- Page table look up — how many memory accesses?



Cache miss, TLB miss, Page fault

Case	\$	TLB	Virtual Memory
L1 \$ Hit	Hit	Hit	No page fault
L1 \$ Miss	Miss	Hit or Miss	May or may not have a page fault
TLB Miss	Miss	Miss	May or may not have a page fault
Page Fault	Miss	Miss	Page fault

TLB.cpp

```
TLB.cpp:2-60 (59 lines)
```

```
//START
```

```
#include<cstdint>
```

```
#include<cstdlib>
```

```
#include<vector>
```

```
#include<algorithm>
```

```
#include "function_map.hpp"
```

```
#include <sys/mman.h>
```

```
template<size_t BYTES>
```

```
struct MM {
```

```
    struct MM* next; // I know that pointers are 8 bytes on this machine.
```

```
    uint64_t junk[BYTES/8 - 1]; // This forces the struct MM to take a up a whole cache line,  
    abolishing spatial locality.
```

```
};
```

```
template<class MM>
```

```
MM * __attribute__((noinline)) miss(MM * start, uint64_t count) {
```

```
    for(uint64_t i = 0; i < count; i++) { // Here's the loop that does this misses. It's very simple
```

```
        start = start->next;
```

```
    }
```

```
    return start;
```

```
}
```

TLB.cpp

```
//
template<size_t BYTES>
uint64_t* TLB(uint64_t * data, uint64_t size, uint64_t arg1) {
    struct MM<BYTES> * array = NULL;
    int r = posix_memalign(reinterpret_cast<void**>(&array), 4096, size);
    if (r == -1) {
        std::cerr << "posix_memalign() failed. Exiting: " << strerror(errno) << "\n";
        exit(1);
    }

    r = madvise(reinterpret_cast<void*>(array), size, MADV_NOHUGEPAGE);
    if (r == -1) {
        std::cerr << "madvise() failed. Exiting: " << strerror(errno) << "\n";
        exit(1);
    }

    std::cout << "array alignment is " << (reinterpret_cast<uintptr_t>(array) % 4096) << "\n";
    std::cout << "array size is " << size/BYTES << " element; " << size << "B\n";

    // This is clever part 'index' is going to determine where the pointers go. We fill it consecutive integers.
    std::vector<uint64_t> index;
    for(uint64_t i = 0; i < size/BYTES; i++) {
        index.push_back(i);
    }
    // Randomize the list of indexes.
    std::random_shuffle(index.begin(), index.end());

    // Convert the indexes into pointers.
    for(uint64_t i = 0; i < size/BYTES; i++) {
        array[index[i]].next = &array[index[(i + 1) % (size/BYTES)]];
    }

    MM<BYTES> * start = &array[0];
    start = miss(start, arg1); // 128 million accesses.
    return reinterpret_cast<uint64_t*>(start); // This is a garbage value, but if we don't return it, the compiler will optimize out the call to
miss.
}
//END
```


Q1: When are we seeing TLB misses?

Address translations are performed through the TLBs: 96-entry 6-way 4-KB-page TLB, 32-entry 4-way 2-MB/4-MB-page TLB, and 8-entry 1-GB-page TLB for loads. A 16-entry TLB for stores server all page sizes. The TLBs are backed by a 2,048-entry second level TLB (STLB), which is shared between code and data requests. STLB misses are sent to the page miss handler (PMH) that allows us to perform up to four page walks in parallel.

If the “working set size” is smaller than $96 \times 4KB = 384KB$, no L1 TLB misses after the first run!

If the “working set size” is smaller than $2048 \times 4KB = 8MB$, no L2 TLB misses after the first run!

Q2: How to measure L2 TLB misses?

Address translations are performed through the TLBs: 96-entry 6-way 4-KB-page TLB, 32-entry 4-way 2-MB/4-MB-page TLB, and 8-entry 1-GB-page TLB for loads. A 16-entry TLB for stores serves all page sizes. The TLBs are backed by a 2,048-entry second level TLB (STLB), which is shared between code and data requests. STLB misses are sent to the page miss handler (PMH) that allows us to perform up to four page walks in parallel.

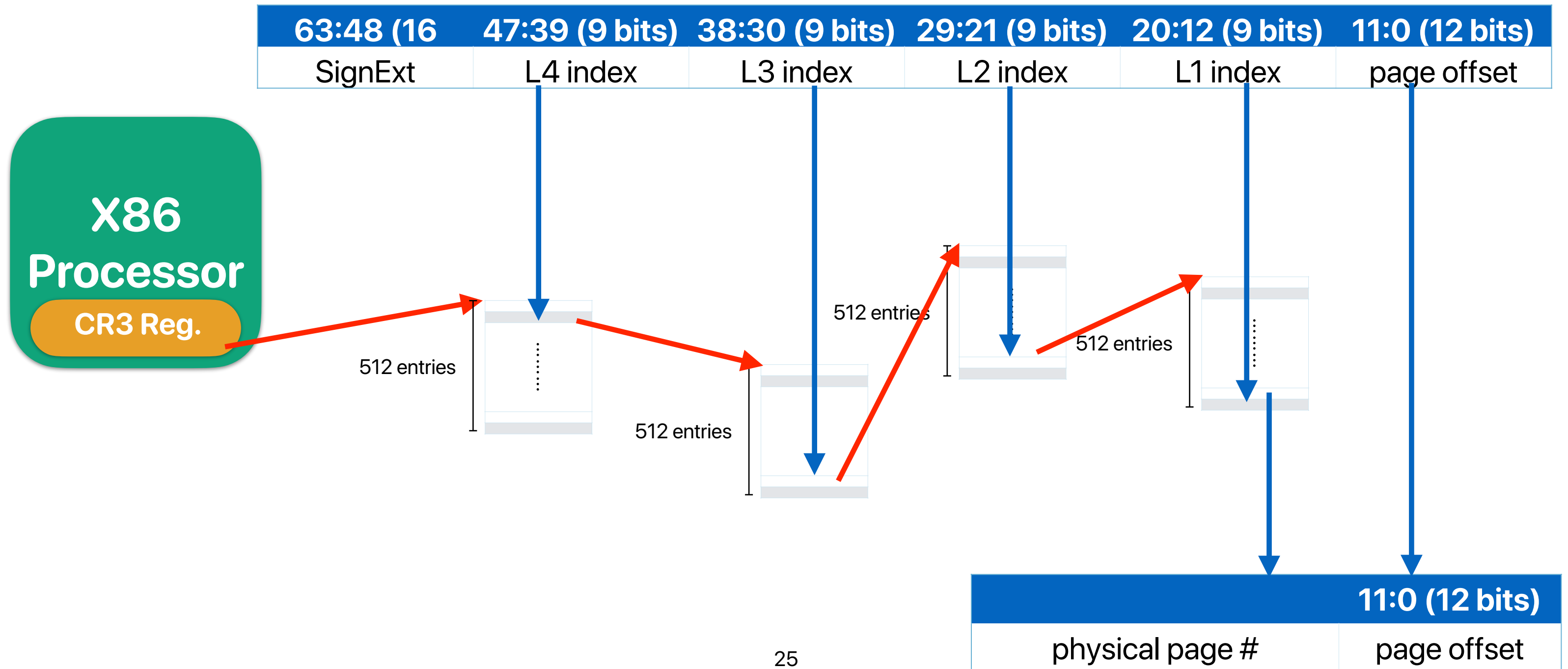
The working set size has to be larger than L2 TLB coverage

It should not go over the L3 capacity — 12MB in our processor

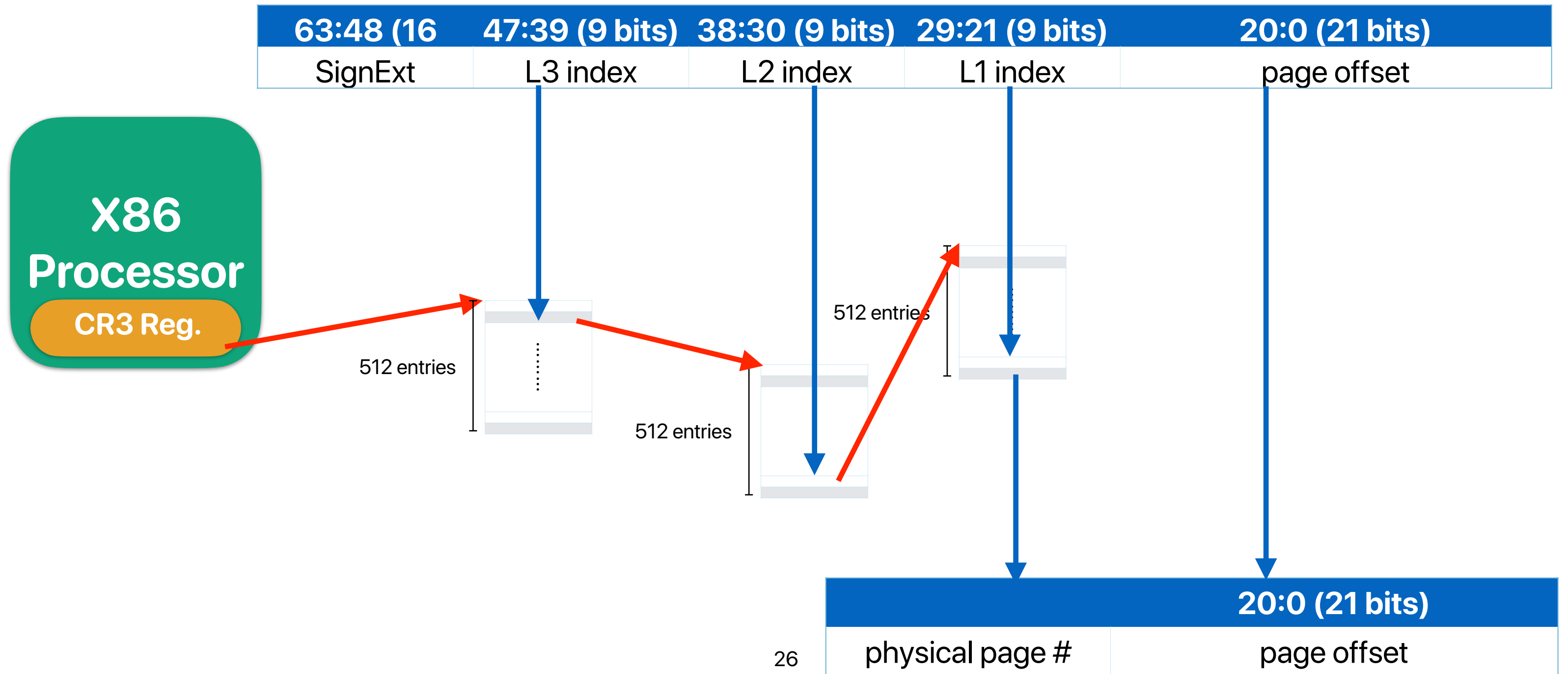
Otherwise, you would measure a lot of L3 miss latency

What happen on a TLB miss?

- Page table look up — how many memory accesses?



Huge page



Pros and cons of huge pages?

- Pros
 - Few page faults! — now only once every 2^{21} Bytes = 2 MB
 - Lower page table lookup overhead — now only 3 levels
- Cons
 - Internal fragmentation (e.g., the data structure does not use the complete 2 MB)
 - May not be able to find consecutive 2MB physical pages if mixed together with 4KB small pages

Q3: How to measure huge pages' overhead?

- Allocate "nodes" each "huge page" (e.g., 2MB)
- Allocate more than the target TLB entries
- Randomly traverse them
- Assume the major latency are coming from memory operations
— cycles divided by memory operations.

```
df = render_csv("TLB2.csv", columns=["function", "size", "arg1",  
"IC", "CPI", "CT", "ET", "L1_dcache_miss_rate",  
"L1_dcache_misses", "L1_dcache_accesses", "Cycles"])  
df["load_latency_cyc"] = df["Cycles"]/df["arg1"]  
df["load_latency_ns"] = df["load_latency_cyc"]*df['CT']*1e9
```

Recap: Working set size

- The portion of memory that the program is currently using
- The total size of data blocks with “temporal locality”
- The total size of data blocks we have to visit before the next reuse of the current block

Recap: conflict misses v.s. capacity misses

- Conflict miss
 - If the miss occurs, and it's not the first time we access this block, and the working set size is smaller than the cache size
 - Solution — avoid the access pattern that creates a "stride" that creates conflicts among blocks
- Capacity miss
 - If the miss occurs, and it's not the first time we access this block, and the working set size is larger than the cache size
 - Solution — reduce the dataset size, reduce the working set size

Recap: Software Optimizations

- Data layout — capacity miss, conflict miss, compulsory miss
- Loop interchange — conflict/capacity miss
- Loop fission — conflict miss — when \$ has limited way associativity
- Loop fusion — capacity miss — when \$ has enough way associativity
- Blocking/tiling — capacity miss, conflict miss
- Matrix transpose (a technique changes layout) — conflict misses
- Prefetching — compulsory misses

Q5: Loop renesting/loop interchange

A

```
for(i = 0; i < ARRAY_SIZE; i++)  
{  
    for(j = 0; j < ARRAY_SIZE; j++)  
    {  
        c[i][j] = a[i][j]+b[i][j];  
    }  
}
```

Loop interchange



B

```
for(j = 0; j < ARRAY_SIZE; j++)  
{  
    for(i = 0; i < ARRAY_SIZE; i++)  
    {  
        c[i][j] = a[i][j]+b[i][j];  
    }  
}
```


Q6: when is loop interchange/re-nesting useless?

What kind of miss does loop interchange eliminate?

Conflict misses

If conflict misses is not the main thing, loop interchange will become useless

Matrix Multiplication — let's consider "b"

```
for(i = 0; i < M; i++) {  
  for(j = 0; j < K; j++) {  
    for(k = 0; k < N; k++) {  
      c[i][j] += a[i][k]*b[k][j];  
    }  
  }  
}
```

- If the row dimension (N) of your matrix is 2048, each row element with the same column index is

$$2048 \times 8 = 16384 = 0x4000$$

away from each other

Each set can store only 12 blocks! So we will start to kick out b[0][0-7], b[1][0-7] ...

	Address	Tag	Index
b[0][0]	0x20000	0x20	0x0
b[1][0]	0x24000	0x24	0x0
b[2][0]	0x28000	0x28	0x0
b[3][0]	0x2C000	0x2C	0x0
b[4][0]	0x30000	0x30	0x0
b[5][0]	0x34000	0x34	0x0
b[6][0]	0x38000	0x38	0x0
b[7][0]	0x3C000	0x3C	0x0
b[8][0]	0x40000	0x40	0x0
b[9][0]	0x44000	0x44	0x0
b[10][0]	0x48000	0x48	0x0
b[11][0]	0x4C000	0x4C	0x0
b[12][0]	0x50000	0x50	0x0
b[13][0]	0x54000	0x54	0x0
b[14][0]	0x58000	0x58	0x0
b[15][0]	0x5C000	0x5C	0x0
b[16][0]	0x60000	0x60	0x0

Now, when we work on c[0][1]

	Address	Tag	Index
b[0][0]	0x20000	0x20	0x0
b[1][0]	0x24000	0x24	0x0
b[2][0]	0x28000	0x28	0x0
b[3][0]	0x2C000	0x2C	0x0
b[4][0]	0x30000	0x30	0x0
b[5][0]	0x34000	0x34	0x0
b[6][0]	0x38000	0x38	0x0
b[7][0]	0x3C000	0x3C	0x0
b[8][0]	0x40000	0x40	0x0
b[9][0]	0x44000	0x44	0x0
b[10][0]	0x48000	0x48	0x0
b[11][0]	0x4C000	0x4C	0x0
b[12][0]	0x50000	0x50	0x0
b[13][0]	0x54000	0x54	0x0
b[14][0]	0x58000	0x58	0x0
b[15][0]	0x5C000	0x5C	0x0

	Address	Tag	Index	
b[0][1]	0x20008	0x20	0x0	Conflict Miss

We're coming back to the block of b[0][0-7].

How big is the working set size?

$$16 \times 64\text{Bytes} = 1024\text{Bytes} < 48\text{KB}$$

Conflict misses! — not capacity misses

Each set can store only 12 blocks! So we will start to kick out b[0][0-7], b[1][0-7] ...

Now, when we work on c[0][1]

	Address	Tag	Index
b[0][0]	0x20000	0x20	0x0
b[1][0]	0x24000	0x24	0x0
b[2][0]	0x28000	0x28	0x0
b[3][0]	0x2C000	0x2C	0x0
b[4][0]	0x30000	0x30	0x0
b[5][0]	0x34000	0x34	0x0
b[6][0]	0x38000	0x38	0x0
b[7][0]	0x3C000	0x3C	0x0
b[8][0]	0x40000	0x40	0x0
b[9][0]	0x44000	0x44	0x0
b[10][0]	0x48000	0x48	0x0
b[11][0]	0x4C000	0x4C	0x0
b[12][0]	0x50000	0x50	0x0
b[13][0]	0x54000	0x54	0x0
b[14][0]	0x58000	0x58	0x0
b[15][0]	0x5C000	0x5C	0x0



	Address	Tag	Index		
b[0][1]	0x20008	0x20	0x0	Conflict	Miss
b[1][1]	0x24008	0x24	0x0	Conflict	Miss
b[2][1]	0x28008	0x28	0x0	Conflict	Miss
b[3][1]	0x2C008	0x2C	0x0	Conflict	Miss
b[4][1]	0x30008	0x30	0x0	Conflict	Miss
b[5][1]	0x34008	0x34	0x0	Conflict	Miss
b[6][1]	0x38008	0x38	0x0	Conflict	Miss
b[7][1]	0x3C008	0x3C	0x0	Conflict	Miss
b[8][1]	0x40008	0x40	0x0	Conflict	Miss
b[9][1]	0x44008	0x44	0x0	Conflict	Miss
b[10][1]	0x48008	0x48	0x0	Conflict	Miss
b[11][1]	0x4C008	0x4C	0x0	Conflict	Miss
b[12][1]	0x50008	0x50	0x0	Conflict	Miss
b[13][1]	0x54008	0x54	0x0	Conflict	Miss
b[14][1]	0x58008	0x58	0x0	Conflict	Miss
b[15][1]	0x5C008	0x5C	0x0	Conflict	Miss

Each set can store only 12 blocks! So we will start to kick out b[0][0-7], b[1][0-7] ...

Now, when we work on c[0][1]

	Address	Tag	Index
b[0][0]	0x20000	0x20	0x0
b[1][0]	0x24000	0x24	0x0
b[2][0]	0x28000	0x28	0x0
b[3][0]	0x2C000	0x2C	0x0
b[4][0]	0x30000	0x30	0x0
b[5][0]	0x34000	0x34	0x0
b[6][0]	0x38000	0x38	0x0
b[7][0]	0x3C000	0x3C	0x0
b[8][0]	0x40000	0x40	0x0
b[9][0]	0x44000	0x44	0x0
b[10][0]	0x48000	0x48	0x0
b[11][0]	0x4C000	0x4C	0x0

	Address	Tag	Index
b[0][1]	0x20008	0x20	0x0

Hit!

If we can limit the access here —

Can you apply the same in Q6?

What value of `-arg1` should result in a very high (e.g., > 95%) hit rate, even with the `x` loop on the inside? Try to reason through the correct value before running any experiments.

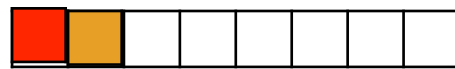
```
uint64_t* x_outside(uint64_t * data, uint64_t size, uint64_t
arg1) {
    uint64_t **matrix = to_2d_array(data, size, arg1);

    for(uint y = 0; y < arg1; y++) {
        for(uint x = 0; x < size/arg1; x++) {
            matrix[x][y] = x;
        }
    }

    return data;
}
```


Q7: 1-D convolution

```
void do_convolution(uint64_t *source,
                   uint64_t *kernel,
                   uint64_t *target,
                   uint64_t source_size,
                   uint64_t kernel_size, uint64_t tile_size) {
    uint64_t target_size = source_size - kernel_size;
    for(register uint64_t i = 0; i < target_size; i++) {
        for(register uint64_t j = 0; j < kernel_size; j++) {
            target[i] += source[i+j] * kernel[j];
            // target[i] = target[i] + source[i+j] * kernel[j];
        }
    }
}
```



target



source



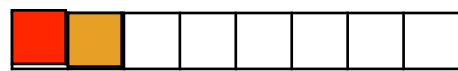
kernel

How many loads? How many stores?

Q8: working set of 1-D convolution

```
void do_convolution(uint64_t *source,
                   uint64_t *kernel,
                   uint64_t *target,
                   uint64_t source_size,
                   uint64_t kernel_size, uint64_t tile_size) {
    uint64_t target_size = source_size - kernel_size;
    for(register uint64_t i = 0; i < target_size; i++) {
        for(register uint64_t j = 0; j < kernel_size; j++) {
            target[i] += source[i+j] * kernel[j];
        }
    }
}
```

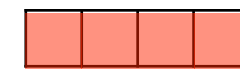
When *i* increments, we will come back to the same source block. How many blocks have visited before that?



target



source



kernel

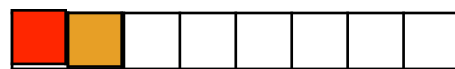
$$\frac{\textit{kernel_size}}{8} \text{ blocks} + \frac{\textit{kernel_size}}{8} \text{ blocks}$$

40

Q8: working set of 1-D convolution

```
void do_convolution(uint64_t *source,
                   uint64_t *kernel,
                   uint64_t *target,
                   uint64_t source_size,
                   uint64_t kernel_size, uint64_t tile_size) {
    uint64_t target_size = source_size - kernel_size;
    for(register uint64_t i = 0; i < target_size; i++) {
        for(register uint64_t j = 0; j < kernel_size; j++) {
            target[i] += source[i+j] * kernel[j];
        }
    }
}
```

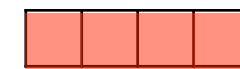
**The previously fetched
source block may be kicked out!**



target



source



kernel

What if I have a 24 KB kernel?

**So we will consistently see misses in
the first access of the block**

Takeaways: Software Optimizations

- Data layout — capacity miss, conflict miss, compulsory miss
- Loop interchange — conflict/capacity miss
- Loop fission — conflict miss — when \$ has limited way associativity
- Loop fusion — capacity miss — when \$ has enough way associativity
- ✓ Blocking/tiling — capacity miss, conflict miss
 - Matrix transpose (a technique changes layout) — conflict misses
 - Prefetching — compulsory misses
 - Data alignments — capacity misses

The first idea

```
for(int64_t i = 0; i < target_size; i++) {  
    for(int64_t jj = 0; jj < kernel_size; jj += tile_size) { // We create a new  
loop variable jj that advanced one chunk at a time.  
        for(int64_t j = jj; j < kernel_size && j < jj + tile_size; j++) { // We  
iterate over the chunk. The more complicated termination  
        // condition keeps us from running off the end of the array  
        target[i] += source[i + j] * kernel[j];  
    }  
}
```

**The tiling is not effective
because...?**

	size	arg1	le_si	function	IC	Cycles	CPI	CT	ET	L1_dcache_miss_rate	L1_dcache_accesses
1	131072	32768	64	convolution	22550921992	4099146202	0.181773	0.250845	1.028250	0.073631	9664503271
2	131072	32768	64	convolution_new_loop	29547281586	5336448357	0.180607	0.250840	1.338596	0.082891	9714951191

More ICs!

Slower!

**Miss
Rate is
high!**

Loop re-nesting/interchange

At least the tiling is now
effective

```
for(int32_t jj = 0; jj < kernel_size; jj += tile_size) { // Move the jj chunk loop
outside
    for(int32_t i = 0; i < target_size; i++) {
        for(int32_t j = jj; j < kernel_size && j < jj + tile_size; j++) {
            target[i] += source[i + j] * kernel[j];
        }
    }
}
```

	size	arg1	le_si	function	IC	Cycles	CPI	CT	ET	L1_dcache_miss_rate	L1_dcache_accesses
1	131072	32768	64	convolution	22550921992	4099146202	0.181773	0.250845	1.028250	0.073631	9664503271
2	131072	32768	64	convolution_new_loop	29547281586	5336448357	0.180607	0.250840	1.338596	0.082891	9714951191
3	131072	32768	64	convolution_tiled	29395066175	4366858628	0.148558	0.250553	1.094130	0.001104	9714790824

More ICs still!

Slower! Lower
miss rate

Reduce the control overhead

apply "loop-unrolling" compiler optimization

```
extern "C"
uint64_t* __attribute__((optimize("unroll-loops"))) convolution_tiled_unrolled(uint64_t * source,
uint64_t source_size,
uint64_t * kernel, uint64_t kernel_size,
uint64_t * target, uint64_t _target_size, int32_t tile_size) {

uint64_t target_size = source_size - kernel_size;
for(int32_t jj = 0; jj < kernel_size; jj += tile_size) { // Move the jj chunk loop outside
for(int32_t i = 0; i < target_size; i++) {
for(int32_t j = jj; j < kernel_size && j < jj + tile_size; j++) {
target[i] += source[i + j] * kernel[j];
}
}
}
}
```

	size	arg1	le_si	function	IC	Cycles	CPI	CT	ET	L1_dcache_miss_rate	L1_dcache_accesses
1	131072	32768	64	convolution	22550921992	4099146202	0.181773	0.250845	1.028250	0.073631	9664503271
2	131072	32768	64	convolution_new_loop	29547281586	5336448357	0.180607	0.250840	1.338596	0.082891	9714951191
3	131072	32768	64	convolution_tiled	29395066175	4366858628	0.148558	0.250553	1.094130	0.001104	9714790824
4	131072	32768	64	convolution_tiled_unrolled	24915161674	3786971187	0.151995	0.251169	0.9488176	0.000942	9714659874

Faster now!

Can we do better?

```
extern "C"
uint64_t* __attribute__((optimize("unroll-loops"))) convolution_tiled_unrolled(uint64_t * source,
uint64_t source_size,
uint64_t * kernel, uint64_t kernel_size,
uint64_t * target, uint64_t _target_size, int32_t tile_size) {

uint64_t target_size = source_size - kernel_size;
for(int32_t jj = 0; jj < kernel_size; jj += tile_size) { // Move the jj chunk loop outside
    for(int32_t i = 0; i < target_size; i++) {
        for(int32_t j = jj; j < kernel_size && i < i + tile_size; j++) {
            target[i] += source[i + j] * kernel[j];
        }
    }
}
return target;
}

if (jj + tile_size > kernel_size) {
    for(int32_t j = jj; j < kernel_size; j++) {
        target[i] += source[i + j] * kernel[j];
    }
} else {
    for(int32_t j = jj; j < jj + tile_size; j++) {
        target[i] += source[i + j] * kernel[j];
    }
}
```

We have to do two checks every time!

Only check `jj+tile_size` once

Further reduce the control/branch overhead

```
uint64_t* __attribute__((optimize("unroll-loops"))) convolution_tiled_split(uint64_t * source, uint64_t source_size,
                                   uint64_t * kernel, uint64_t kernel_size,
                                   uint64_t * target, uint64_t _target_size, int32_t tile_size) {
    uint64_t target_size = source_size - kernel_size;
    int32_t real_tile_size = tile_size/8 * 8; // this clears the low 3 bits. Check the assembly!
    assert(tile_size>=8);

    for(int32_t jj = 0; jj < kernel_size; jj += real_tile_size) { // Move the jj chunk loop outside
        for(int32_t i = 0; i < target_size; i++) {
            if (jj + real_tile_size > kernel_size) {
                for(int32_t j = jj; j < kernel_size; j++) {
                    target[i] += source[i + j] * kernel[j];
                }
            } else {
                for(int32_t j = jj; j < jj + real_tile_size; j++) {
                    target[i] += source[i + j] * kernel[j];
                }
            }
        }
    }
    return target;
}
```

	size	arg1	arg2	arg3	function	IC	Cycles	CPI	CT	ET	L1_dcache_miss_rate	L1_dcache_accesses
1	131072	32768	64		convolution	22550921992	4099146202	0.181773	0.250845	1.028250	0.073631	9664503271
2	131072	32768	64		convolution_new_loop	29547281586	5336448357	0.180607	0.250840	1.338596	0.082891	9714951191
3	131072	32768	64		convolution_tiled	29395066175	4366858628	0.148558	0.250553	1.094130	0.001104	9714790824
4	131072	32768	64		convolution_tiled_unrolled	24915161674	3786971187	0.151995	0.251169	0.9488176	0.000942	9714659874
5	131072	32768	64		convolution_tiled_split	16359042414	3252245650	0.198804	0.250875	0.815907	0.001299	9714595552

IC drops a lot!

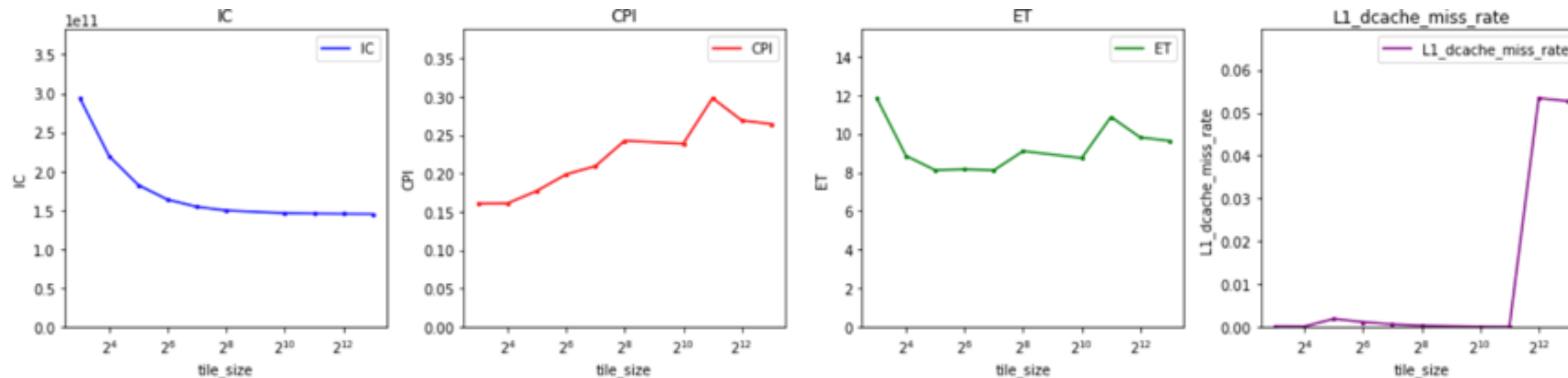
Significant improvement

Selecting the right tile size!

	size	rep	arg1	tile_size	function	IC	Cycles	CPI	CT	ET	_dcache_miss_ra
1	131072	1	32768	8	convolution_tiled_split	293954351884	47268351256	0.160802	0.250537	11.842483	0.000183
2	131072	1	32768	16	convolution_tiled_split	219458852083	35342730839	0.161045	0.250544	8.854903	0.000140
3	131072	1	32768	32	convolution_tiled_split	182212269246	32299455989	0.177263	0.251226	8.114475	0.001899
4	131072	1	32768	64	convolution_tiled_split	163589599579	32526336632	0.198890	0.250926	8.162029	0.001160
5	131072	1	32768	128	convolution_tiled_split	154278148344	32315039644	0.209460	0.250937	8.109041	0.000628
6	131072	1	32768	256	convolution_tiled_split	149623856171	36303704199	0.242633	0.250720	9.102061	0.000320
7	131072	1	32768	1024	convolution_tiled_split	146131592161	34898560301	0.238816	0.250537	8.743379	0.000084
8	131072	1	32768	2048	convolution_tiled_split	145552912507	43387111422	0.298085	0.250538	10.870136	0.000061
9	131072	1	32768	4096	convolution_tiled_split	145260546766	39083417418	0.269057	0.250995	9.809729	0.053440
10	131072	1	32768	8192	convolution_tiled_split	145114375483	38371619605	0.264423	0.251070	9.633955	0.052744

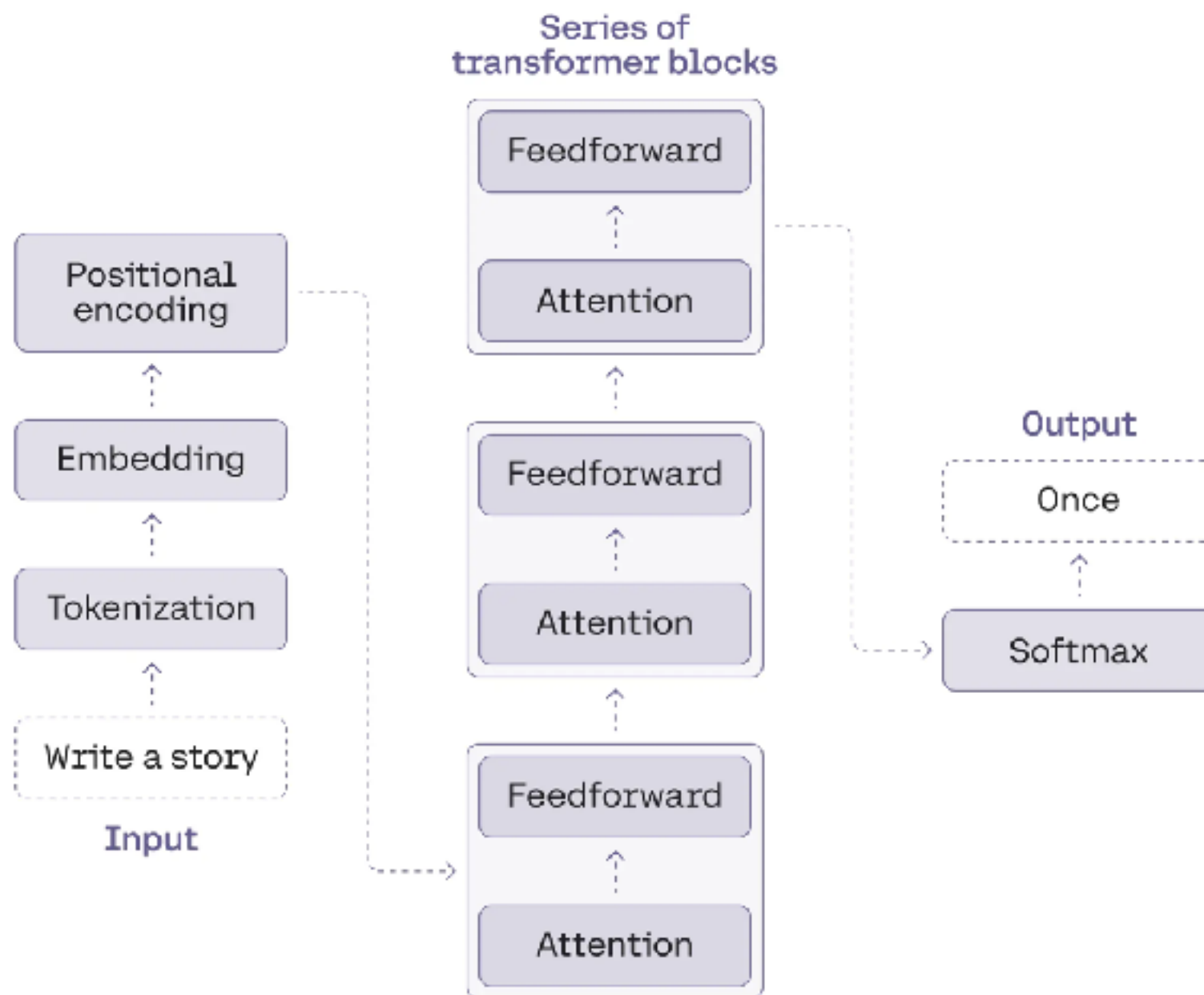
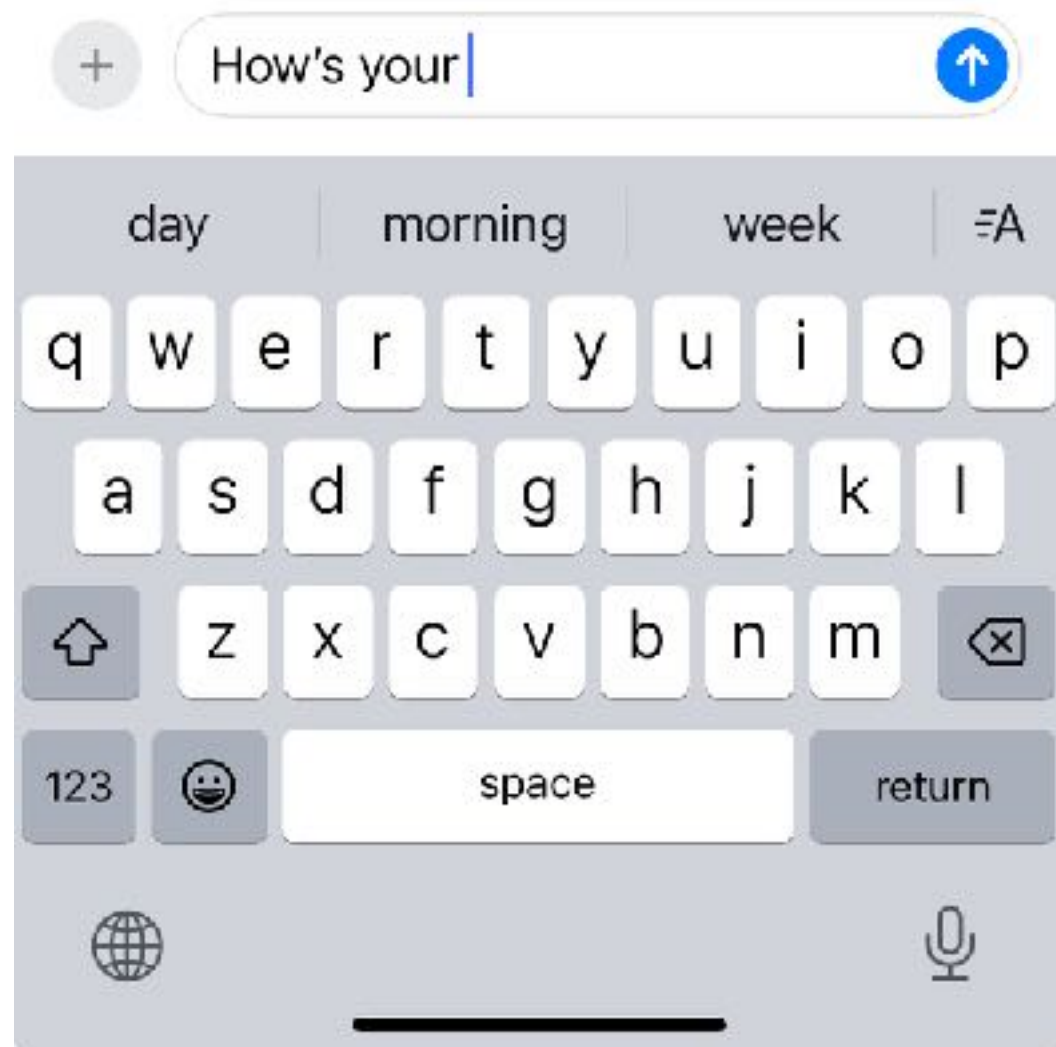
Sweet spot

Trade-offs between IC and miss rate!

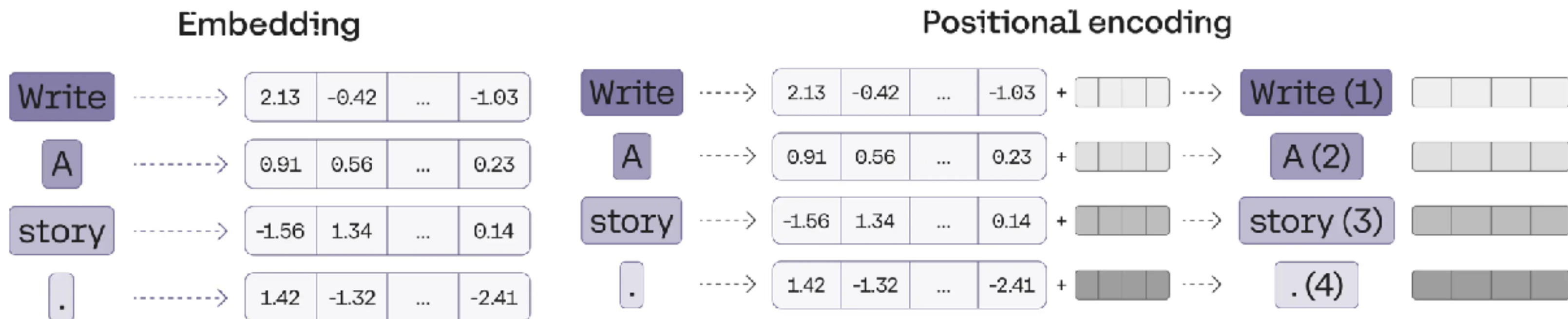


Programming assignment 2

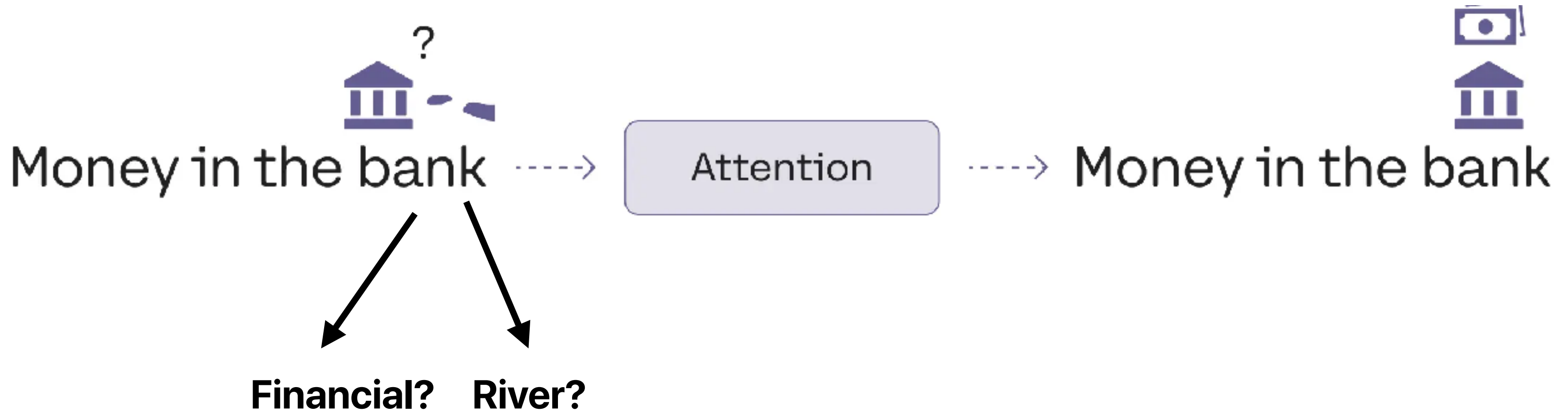
Transformer



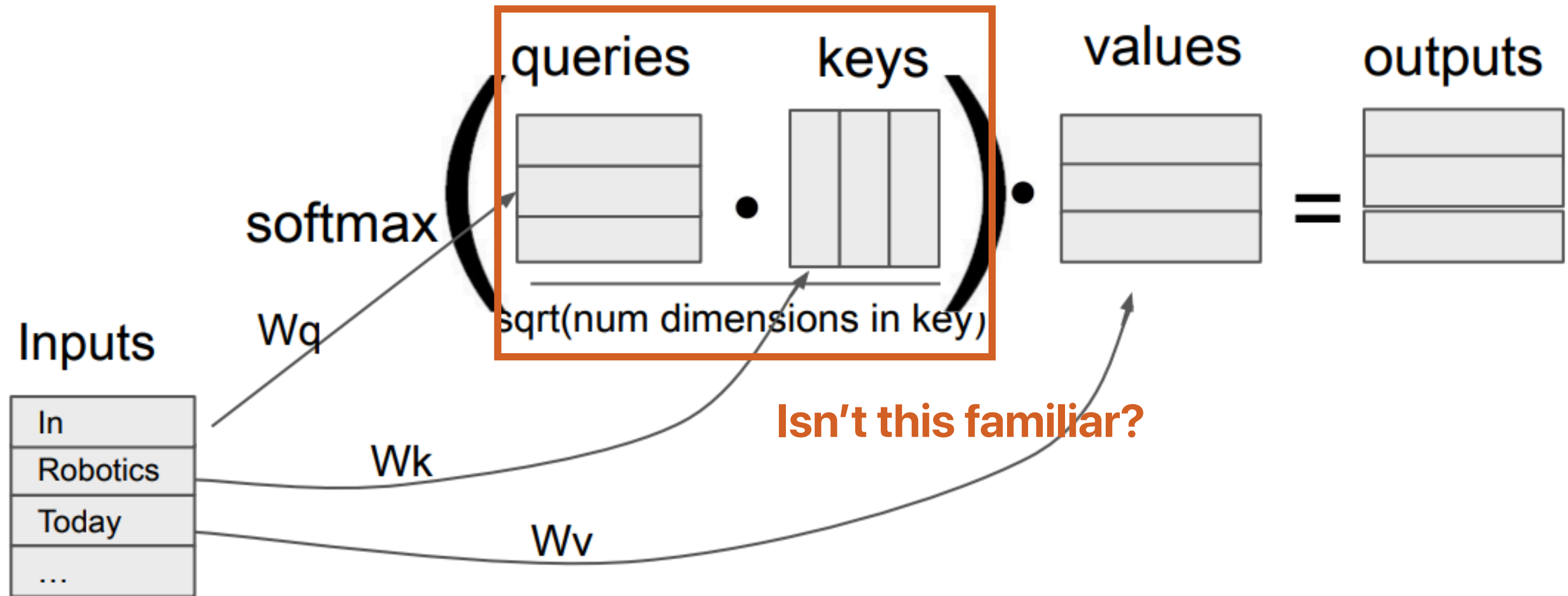
Encoding in Transformer



Attention



Softmax



Things to do

```
void matmul_backward(float* dinp, float* dweight, float* dbias,
                    const float* dout, const float* inp, const float* weight,
                    int B, int T, int C, int OC) {

    for (int b = 0; b < B; b++) {
        for (int t = 0; t < T; t++) {
            const float* dout_bt = dout + b * T * OC + t * OC;
            float* dinp_bt = dinp + b * T * C + t * C;
            for (int o = 0; o < OC; o++) {
                const float* wrow = weight + o * C;
                float d = dout_bt[o];
                for (int i = 0; i < C; i++) {
                    dinp_bt[i] += wrow[i] * d;
                }
            }
        }
    }

    // backward into weight/bias, parallelize over output channels OC
    for (int o = 0; o < OC; o++) {
        for (int b = 0; b < B; b++) {
            for (int t = 0; t < T; t++) {
                const float* dout_bt = dout + b * T * OC + t * OC;
                const float* inp_bt = inp + b * T * C + t * C;
                float* dwrow = dweight + o * C;
                float d = dout_bt[o];
                if (dbias != NULL) { dbias[o] += d; }
                for (int i = 0; i < C; i++) {
                    dwrow[i] += inp_bt[i] * d;
                }
            }
        }
    }
}
```

- Blocking/tiling to improve cache miss rate
 - Careful not to cause increase in ICs
- Loop fission? Fusion?
- Our goal is 1.1x on Gradescope server
 - The same architecture as the lab machine
 - The server uses Xeon Platinum 8275CL, so the speedup of 1.15x on the lab machine is about right

Computer
Science &
Engineering

142L

つづく

